

Question 1: Top 3 suburbs by transaction count + monthly 2022 plot

Question 2: The 3 most important keywords in `description` impacting price

Question 3: Correlation of price and land size by (top-3 suburb × property type)

Question 4: Top 5 capital gains (resold within 5 years)

Question 5: Properties that could not realistically exist

Question 6: Predicted July 2026 price for Chris's house in 6 suburbs

35507071_TaskC_EDA

[Code ▼](#)

Wanting Echo Zhao

30 May, 2026

[Hide](#)

```
# Load necessary packages
library(tidyverse)
library(dplyr)
library(tidyr)
library(ggplot2)
library(lubridate)
library(stringr)
library(naniar)
library(data.table)
library(gt)

prop <- read_csv("TaskC_property_victoria.csv")

# overview the data
cat("row: ", nrow(prop), "column:", ncol(prop))
```

```
## row: 418390 column: 12
```

[Hide](#)

```
# see the type of each attribute
str(prop)
```

```
## spc_tbl_ [418,390 × 12] (S3: spec_tbl_df/tbl_df/tbl/data.frame)
## $ ID : num [1:418390] 1.28e+08 1.23e+08 2.02e+08 2.03e+08 1.40e+08 ...
## $ postcode : num [1:418390] 3095 3074 3978 3753 3197 ...
## $ suburb : chr [1:418390] "Eltham" "Thomastown" "Clyde North" "Beveridge" ...
## $ sold_time : Date[1:418390], format: "2018-07-04" "2016-07-16" ...
## $ sold_price : num [1:418390] 1900000 424000 309000 NA 901000 385000 710000 NA 42500
0 650000 ...
## $ address : chr [1:418390] "10 Sweeneys Lane, Eltham, Vic 3095" "2 Banksia Avenu
e, Thomastown, Vic 3074" "74 Yeungroon Boulevard, Clyde North, Vic 3978" "Lot 730, Radiata
Street, Beveridge, Vic 3753" ...
## $ beds : num [1:418390] 7 3 NA NA 3 1 3 3 1 3 ...
## $ baths : num [1:418390] 4 1 NA NA 1 1 2 2 1 2 ...
## $ parking : num [1:418390] 2 2 NA NA 3 1 2 3 1 1 ...
## $ area : num [1:418390] 4322 659 448 325 638 ...
## $ property_type: chr [1:418390] "house" "house" "residential-land" "residential-land"
...
## $ description : chr [1:418390] "Set in around 4323 m2 of rolling lawns surrounded by
gardens landscaped by Ben Harris, this is family living on"| __truncated__ "AUCTION THIS S
ATURDAY AT 1PM<br/>Perfectly located in the popular and much sought after Thomastown West
locale "| __truncated__ "Located in the developing St Germain Estate of Clyde North is thi
s 447 sq mtr block. With a frontage of 14mtrs "| __truncated__ "Fully Fenced<br/>$10,000 D
eposit" ...
## - attr(*, "spec")=
## .. cols(
## .. ID = col_double(),
## .. postcode = col_double(),
## .. suburb = col_character(),
## .. sold_time = col_date(format = ""),
## .. sold_price = col_double(),
## .. address = col_character(),
## .. beds = col_double(),
## .. baths = col_double(),
## .. parking = col_double(),
## .. area = col_double(),
## .. property_type = col_character(),
## .. description = col_character()
## .. )
## - attr(*, "problems")=<externalptr>
```

Question 1: Top 3 suburbs by transaction count + monthly 2022 plot

Mind map

- Find the three suburbs with the highest number of transactions *across all years*.
- Then plot the monthly transaction counts during 2022 for those suburbs.
- If *Toorak* is not already in the top three, include it as a fourth line on the plot.

[Hide](#)

```
# Compute NA value
miss_var_cumsum(prop)
```

```
## # A tibble: 12 × 3
##   variable      n_miss n_miss_cumsum
##   <chr>         <int>         <int>
## 1 ID             21             21
## 2 postcode       21             42
## 3 suburb         21             63
## 4 sold_time     5218            5281
## 5 sold_price    50425            55706
## 6 address       6158            61864
## 7 beds         19884            81748
## 8 baths         19884           101632
## 9 parking       19884           121516
## 10 area          28           121544
## 11 property_type 21           121565
## 12 description   26           121591
```

Hide

```
# There is 21 miss data in `suburb`

# Find the top 3 suburbs by transaction count
prop_clean <- prop |>
  filter(!is.na(suburb))

# Compute the count of transactions of each suburb, then sort descently. sclice top 3
sub_count <- prop_clean |>
  count(suburb, sort = TRUE, name = "transation_count")

# show top 10
head(sub_count, 10)
```

```
## # A tibble: 10 × 2
##   suburb      transation_count
##   <chr>          <int>
## 1 Toorak         3818
## 2 Melbourne     3556
## 3 Clyde North   2090
## 4 Mornington    2031
## 5 Prahran       2031
## 6 Preston       2025
## 7 Tarneit       2023
## 8 Officer       2022
## 9 Sunbury       2011
## 10 Truganina    2008
```

Hide

```
# Top 1 is Toorak, so there is no need to add it individually for question 1
# extract top 3
top3 <- sub_count$suburb[1:3]
top3
```

```
## [1] "Toorak"      "Melbourne"   "Clyde North"
```

The top three suburbs are **Toorak (3,818), Melbourne (3,556), and Clyde North (2,090)**. Toorak is already #1, so the plot below contains exactly those three lines (no extra Toorak line is needed).

Hide

```
# Build a (suburb + month) count table for 2022 only.

monthly_2022 <- prop_clean |>
  filter(year(sold_time) == 2022,
         suburb %in% top3) |>
  count(suburb, month = month(sold_time), name = "transation_counts") |>
  arrange(suburb, month)

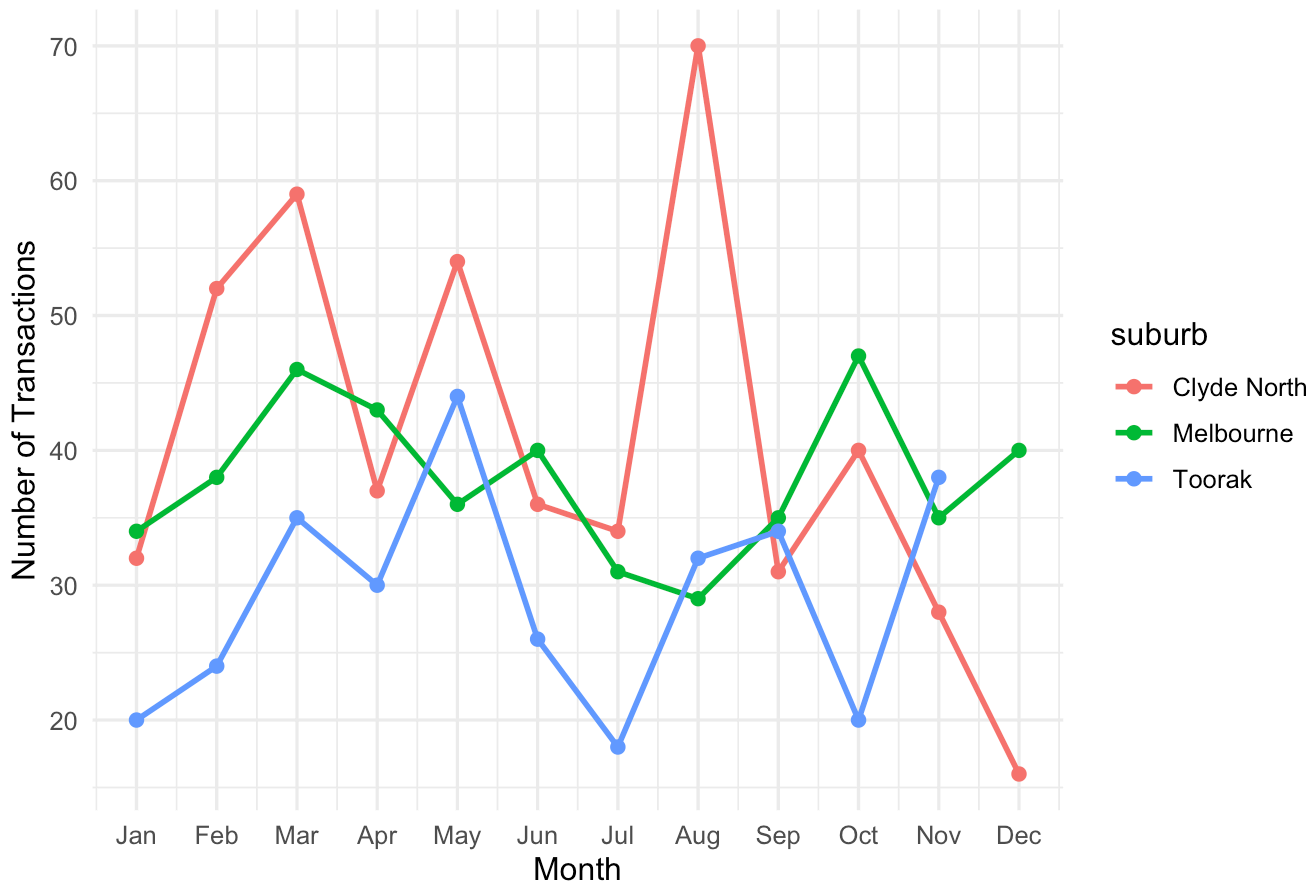
# Transfer long to wide format
monthly_2022 |>
  pivot_wider(names_from = month, values_from = transation_counts)
```

```
## # A tibble: 3 × 13
##   suburb   `1`   `2`   `3`   `4`   `5`   `6`   `7`   `8`   `9`  `10`  `11`  `12`
##   <chr> <int> <int> <int> <int> <int> <int> <int> <int> <int> <int> <int> <int>
## 1 Clyde...    32    52    59    37    54    36    34    70    31    40    28    16
## 2 Melbo...    34    38    46    43    36    40    31    29    35    47    35    40
## 3 Toorak     20    24    35    30    44    26    18    32    34    20    38    NA
```

Hide

```
# Plot it
ggplot(monthly_2022, aes(x = month, y = transation_counts, colour = suburb, group = suburb)) +
  geom_line(linewidth = 1) +
  geom_point(size = 2) +
  scale_x_continuous(breaks = 1:12, labels = month.abb) + # 12 months and show as abbr, saving space and looks tidier.
  labs(title = "Monthly property transactions in 2022",
       x = "Month",
       y = "Number of Transactions") +
  theme_minimal(base_size = 12)
```

Monthly property transactions in 2022



Interpretation of plot

Clyde North peaks in August 2022 (70 transactions), the biggest single suburb-month in the chart, before falling sharply at the end of year. Melbourne is the steadiest of the three. Toorak shows the lowest volume of the three and has **zero** December 2022 transactions — likely a data-collection gap rather than a market collapse.

Question 2: The 3 most important keywords in description impacting price

[Hide](#)

```
library(Matrix) # sparse document-term matrix
library(glmnet) # LASSO

# STEP1: 10% random sample, keeping only rows with a price and a description.
set.seed(123)
samp <- prop |>
  filter(!is.na(sold_price), !is.na(description)) |>
  slice_sample(prop = 0.1)

cat("Sample size:", nrow(samp), "rows\n")
```

```
## Sample size: 36796 rows
```

Hide

```
# Having a look at the top 10 row of sample$description,
# Figure out what kind of messy should we clean
# head(samp$description, 10)

# STEP2: Clean the description text:
clean_text <- samp$description |>
  str_replace_all("<[^\>]+>", " ") |> # strip HTML tags like <br/>
  str_to_lower() |> # lower case
  str_replace_all("[^a-z ]+", " ") |> # keep pure letters. Drop all numbers
  str_squish() # collapse multiple space, so later easier to split text.
```

Reference: Week 6 (<https://learning.monash.edu/course/view.php?id=41099§ion=28>) Map-reduce - data table (<https://cran.r-project.org/web/packages/data.table/vignettes/datatable-intro.html>)

Hide

```
# STEP3: Tokenise and explode into a long (doc, term) table using data.table
# a 10% sample is still ~37k docs and 28k+ unique terms).
tokens <- str_split(clean_text, " ")
dtm_long <- data.table(doc = rep(seq_along(tokens), lengths(tokens)),
                      term = unlist(tokens))

# STEP4: Remove various stopword

# 1) Grab general English stopwords (Snowball is a reliable, standard lexicon)
general_stopwords <- stopwords::stopwords(language = "en", source = "snowball")

# 2) Define your specific technical and domain-specific noise sequences
html_leftovers <- c("amp", "br", "lt", "gt", "quot", "apos")

real_estate_terms <- c(
  "also", "new", "one", "two", "three", "four", "five",
  "feature", "features", "property", "home", "house", "bed", "bedroom", "bedrooms",
  "bath", "bathroom", "bathrooms", "room", "rooms", "living", "please", "including",
  "call", "inspection", "view", "contact", "auction", "sale"
)

# 3) Combine them into a single, deduplicated vector
stopwords_en <- unique(c(general_stopwords, html_leftovers, real_estate_terms))

# 4) Filter the data.table
dtm_long <- dtm_long[term != "" & nchar(term) >= 3]
dtm_long <- dtm_long[!term %in% stopwords_en]

nrow(dtm_long) # 4029090
```

```
## [1] 4029090
```

Hide

```
# STEP5: Vocabulary filter: keep only terms that appear in at least 1% of docs.
# Filter: Rare/unique words make the model unstable; very common words don't help either.
min_df <- ceiling(0.01 * nrow(samp))
keep_terms <- dtm_long[, .(df = uniqueN(doc)), by = term][df >= min_df, term]
dtm_long <- dtm_long[term %in% keep_terms]
dtm_long <- unique(dtm_long, by = c("doc", "term")) # collapse to PRESENCE
cat("Vocabulary size after filtering:", length(keep_terms), "terms\n")
```

```
## Vocabulary size after filtering: 1500 terms
```

Hide

```
# STEP6: Build the sparse document-term matrix
term_levels <- sort(unique(dtm_long$term))
dtm <- sparseMatrix(
  i = dtm_long$doc,
  j = match(dtm_long$term, term_levels),
  x = 1,
  dims = c(nrow(samp), length(term_levels)),
  dimnames = list(NULL, term_levels)
)
dim(dtm)
```

```
## [1] 36796 1500
```

Hide

```
# 7) Fit LASSO with cross-validated lambda.
y <- log(samp$sold_price)
fit <- cv.glmnet(dtm, y, alpha = 1, nfolds = 5)

# Pull non-zero coefficients at lambda.1se (the more conservative pick:
# the largest lambda whose error is within 1 SE of the minimum).
co <- coef(fit, s = "lambda.1se")
coef_tbl <- tibble(term = rownames(co)[-1], # drop intercept
                  coef = as.numeric(co)[-1]) |>
  filter(coef != 0) |>
  arrange(desc(abs(coef)))

# Top 20 for context, then the answer (top 3)
head(coef_tbl, 20)
```

```
## # A tibble: 20 × 2
##   term      coef
##   <chr>    <dbl>
## 1 toorak    0.289
## 2 apartment -0.259
## 3 melton   -0.208
## 4 acres     0.200
## 5 hydronic  0.192
## 6 camberwell 0.182
## 7 marble    0.153
## 8 cellar    0.150
## 9 fireplaces 0.145
## 10 communal -0.136
## 11 doncaster 0.132
## 12 miele     0.130
## 13 stca      0.128
## 14 ofp       0.126
## 15 townhouses 0.121
## 16 victorian 0.120
## 17 butler    0.120
## 18 affordable -0.119
## 19 frankston -0.117
## 20 period    0.111
```

Hide

```
top3_keywords <- head(coef_tbl, 3)
top3_keywords
```

```
## # A tibble: 3 × 2
##   term      coef
##   <chr>    <dbl>
## 1 toorak    0.289
## 2 apartment -0.259
## 3 melton   -0.208
```

LASSO (<https://www.r-bloggers.com/2020/05/quick-tutorial-on-lasso-regression-with-example/>)

Answer

The three keywords with the highest coefficients are: toorak (coef: 0.288), apartment (coef: -0.259), melton (coef: -0.208)

- toorak (positive): when “toorak” appears in the description, the predicted price rises.
- apartment (negative): the word “apartment” in the description is associated with a ~26% *lower* predicted price.
- melton (negative): “melton (https://en.wikipedia.org/wiki/Melton,_Victoria)” are associated with a ~21% lower price.

Insights

Some of the top words (toorak , camberwell , melton) are *suburb names*. They function as legitimate description keywords, but they partly proxy for the suburb itself. The other items in the Some keywords in top 20, such as marble , cellar , fireplaces , hydronic , victorian , miele , correspond more directly to

property *features* and align human's intuition that luxury fittings drive prices up. This also can be tell by words like communal, affordable with negative coefficient, which indicating that the presence of these words suggests lower property prices.

Question 3: Correlation of price and land size by (top-3 suburb × property type)

Mind map

- Get each suburb among the top 3 suburbs identified in Q1 - Extract specific columns
- sold_price and area - sold_price remove NA - area remove minus value and NA. so I filter on area > 0 (this drops both NA and -1).
- for each suburb and for each property type suggest group.

Because small sample sizes (like only 2 observations) artificially force a perfect correlation of, flagging low n_obs helps readers know which results are reliable and which are misleading.

obs: www.wisdomlib.org. (2025, August 18). Number of observations: Significance and symbolism. <https://www.wisdomlib.org/concept/number-of-observations> (<https://www.wisdomlib.org/concept/number-of-observations>)

Hide

```
top3_suburbs <- c("Toorak", "Melbourne", "Clyde North")
types_wanted <- c("house", "unit", "townhouse", "apartment")

prop_q3 <- prop |>
  filter(suburb %in% top3_suburbs,
         property_type %in% types_wanted,
         !is.na(sold_price),
         !is.na(area), area > 0)

# How many suburb and property
prop_q3 |>
  count(suburb, property_type) |>
  pivot_wider(names_from = property_type, values_from = n, values_fill = 0)
```

```
## # A tibble: 3 × 5
##   suburb      house townhouse unit apartment
##   <chr>      <int>      <int> <int>      <int>
## 1 Clyde North    808         43     1         0
## 2 Melbourne      0          0     2        139
## 3 Toorak        334          9     0         11
```

Hide

```
# Compute the correlation per group
cor_tbl <- prop_q3 |>
  group_by(suburb, property_type) |>
  summarise(n_obs = n(),
            correlation = if(n() >= 2) cor(sold_price, area) else NA_real_,
            .groups = "drop") |>
  filter(!is.na(correlation)) |>
  arrange(suburb, property_type)
```

cor_tbl

```
## # A tibble: 7 × 4
##   suburb      property_type n_obs correlation
##   <chr>      <chr>          <int>      <dbl>
## 1 Clyde North house           808      0.323
## 2 Clyde North townhouse        43      0.483
## 3 Melbourne apartment        139      0.0761
## 4 Melbourne unit                2      1
## 5 Toorak     apartment         11      0.899
## 6 Toorak     house           334      0.706
## 7 Toorak     townhouse          9      0.391
```

Findings

Cells with reasonable sample sizes(n_obs):

- **Toorak / house** (n = 334, $r \approx 0.71$): strong positive — in this suburb, bigger land translates into substantially higher prices.
- **Clyde North / house** (n = 808, $r \approx 0.32$): moderate positive — outer-suburb new builds, where land matters but standardised house specs flatten the relationship somewhat.
- **Melbourne / apartment** (n = 139, $r \approx 0.08$): essentially no relationship — for inner-city apartments, location, floor, and finishes dominate; the recorded land area is barely informative.

Cells with very few observations — **Melbourne / unit** (n = 2), **Toorak / townhouse** (n = 9), **Toorak / apartment** (n = 11) are kept for completeness but should be read with caution; $r = 1$ for Melbourne / unit is a mathematical artefact of having only two data points, not a meaningful finding.

Note: There are no apartment recorded in Clyde North, 0 house and townhouse in Melbourn, and non unit in Toorak.

Question 4: Top 5 capital gains (resold within 5 years)

Mind Map

- Which properties means lock property_type
- highest price increases , as far as I understand, there are two ways to measure. One is to compare the price increase by the gap between first sold_price and its maximum value. Second is to compare

popular.

- exclude properties where the time between the first and last sale exceeds five years. means mutate a new column, computing the duration, filter ones exceed 5.
- List the top five: Sort and
- address, capital gain, and the duration between the first and last sale. :select these columns.

Challenges and Solutions

1. Only sold_time, the ID is transaction number instead of identity of each property. property with same postcode and address should be same one, and they can be encode with a new "ID". The reason of why using "address + postcode" as key, even address include postcode is to uniquely identify properties safely, in case there are typo.
2. Extreme value handling. In order to avoid impaction by outliers. control the range of sold_price

Hide

```
head(sort(prop$sold_price),5)
```

```
## [1] 1 1 1 1 270
```

Hide

```
tail(sort(prop$sold_price),5)
```

```
## [1] 32000000 80000088 80000088 320000000 1010000000
```

Using chunk above you can notice the minimum price would be 1 and the top two (\$1.01B and \$320M) are implausible. I therefore restrict to prices in the range **$\$50,000 \leq \text{sold_price} \leq \$50,000,000$** before computing capital gains. This keeps the genuine premium Toorak sales (up to $\approx \$32\text{M}$) while removing the obvious typos.

Hide

```

# 1) Filter to rows with valid address, plausible price, and valid date.
ok <- prop |>
  filter(!is.na(address), address != "",
         !is.na(sold_price), sold_price >= 50000, sold_price <= 50e6,
         !is.na(sold_time))

# 2) Synthesis filtered properties using data.table.

ok_dt <- as.data.table(ok)

per_property <- ok_dt[, .(
  n_sales      = .N,
  first_date   = min(sold_time),    # column first_date
  last_date    = max(sold_time),    # column last_date
  first_price  = sold_price[which.min(sold_time)], # column first_price
  last_price   = sold_price[which.max(sold_time)]  # column last_price
), by = .(address, postcode) # double insurance - filter the same property
][n_sales >= 2 & last_date > first_date] # at least two different-day sales

# 3) Derive capital gain and duration in years
per_property$gain <- per_property$last_price - per_property$first_price
per_property$duration_years <- as.numeric(per_property$last_date - per_property$first_date) / 365

# 4) Apply the ≤ 5-year filter and pick the top 5

top5_q4 <- per_property |>
  as_tibble() |> # output should be a table
  filter(duration_years <= 5) |>
  arrange(desc(gain)) |>
  slice_head( n = 5 ) |>
  mutate(duration_years = round(duration_years,2)) |>
  select(address, gain, duration_years,
         first_date, first_price, last_date, last_price)

top5_q4

```

```

## # A tibble: 5 × 7
##   address      gain duration_years first_date first_price last_date  last_price
##   <chr>      <dbl>         <dbl> <date>         <dbl> <date>         <dbl>
## 1 9 Whernsid... 1.10e7           4.35 2016-11-08    11000000 2021-03-13    22000000
## 2 33 Grandvi... 6.68e6           4.08 2016-09-19     9320000 2020-10-17    16000000
## 3 4 Woorigol... 5.43e6           1.48 2014-05-03     3075000 2015-10-26     8500000
## 4 6-8 Glenis... 5.09e6           2.35 2019-12-17     2510000 2022-04-21     7600000
## 5 8 Jamieson... 5    e6           4.18 2011-05-16     1100000 2015-07-18     6100000

```

Answer

Table (property_table_q4.png)

Question 5: Properties that could not realistically exist

Mindmap

- Removing negative values from area.

[Hide](#)

```
# Drop negative area values per the spec. (NA area is allowed to stay –  
# the instruction only says to remove *negative* values.)  
prop_q5 <- prop |> filter(is.na(area) | area >= 0)  
cat("Rows after removing negative area:", nrow(prop_q5), "\n")
```

```
## Rows after removing negative area: 191168
```

Implausible senarios

[Hide](#)

```
# 1) Tiny area for a multi-bedroom  
row1 <- prop_q5 |>  
  filter(area>=1, area <=3, beds >= 3) |>  
  select(ID, address, beds, baths, parking, area, property_type) |>  
  head(1)
```

[Hide](#)

```
# 2) Implausibly large parking space  
row2 <- prop_q5 |>  
  filter(parking >= 100) |>  
  select(ID, address, beds, baths, parking, area, property_type) |>  
  arrange(desc(parking)) |>  
  head(1)
```

[Hide](#)

```
# 3) Implausibly large bedroom counts in a single house  
row3 <- prop_q5 |>  
  filter(beds >= 20) |>  
  select(ID, address, beds, baths, parking, area, property_type) |>  
  arrange(desc(beds)) |>  
  head(1)
```

[Hide](#)

```
# 4) Implausibly large bathroom counts in a single house
row4 <- prop_q5 |>
  filter(baths >= 18, area <= 5) |>
  select(ID, address, beds, baths, parking, area, property_type) |>
  arrange(desc(baths)) |>
  head(1)
```

Hide

```
# 5) Implausibly large bathroom counts more than bedroom
row5 <- prop_q5 |>
  filter(beds == 0, baths >= 3) |>
  select(ID, address, beds, baths, parking, area, property_type) |>
  arrange(desc(baths)) |>
  head(1)
```

Hide

```
implusiable_table <- bind_rows(row1, row2, row3, row4, row5)

# use `implusiable_table` to generate a final table like assignment requires
summary_table <- implusiable_table |>
  mutate(
    Feature = c("area", "parking", "beds", "baths", "beds + baths"),
    Reason = c(
      "A house on ~1 m2 of land cannot physically hold 3+ bedrooms.",
      "11,951 parking spaces for a suburban house (next-highest in the data is 200).",
      "30 bedrooms in a single house; the address field is also corrupted.",
      "20 bathrooms recorded on a property of only ~1 m2 of land.",
      "0 bedrooms but 3+ bathrooms - impossible for a residential house."
    )
  ) |>
  select(Feature, Reason, Address = address, `Property ID` = ID)

knitr::kable(summary_table) # renders as a proper table in the knit
```

Feature	Reason	Address	Property ID
area	A house on ~1 m2 of land cannot physically hold 3+ bedrooms.	37 Beatties Road, Koo Wee Rup, Vic 3981	107405202
parking	11,951 parking spaces for a suburban house (next-highest in the data is 200).	50 Beverley Road, McCrae, Vic 3938	126712114
beds	30 bedrooms in a single house; the address field is also corrupted.	83 Bedrooms, 34 Bathrooms, Burwood, Vic 3125	118443975
baths	20 bathrooms recorded on a property of only ~1 m2 of land.	18 Park Road, Warburton, Vic 3799	124748374
beds + baths	0 bedrooms but 3+ bathrooms - impossible for a residential house.	38 Fitzwilliam Drive, Doreen, Vic 3754	132715294

Question 6: Predicted July 2026 price for Chris's house in 6 suburbs

Mindmap

- Near shopping center and primary school
- 4 bedrooms and 2 bathroom
- in six suburbs—Mulgrave, Vermont South, Doncaster East, Rowville, Glen Waverley, and Wheelers Hill.
- output: price for July 2026, Note: the sold time range is ["2010-01-01", "2023-04-14"]. Hence need predict 3 years later.

[Hide](#)

```
target_suburbs <- c("Mulgrave", "Vermont South", "Doncaster East", "Rowville", "Glen Waverley", "Wheelers Hill")

# filter houses in these 6 suburbs
prop_q6 <- prop |>
  filter(property_type == "house",
         suburb %in% target_suburbs,
         !is.na(sold_price),
         !is.na(area), area>0,
         !is.na(beds), !is.na(baths),
         !is.na(sold_time), !is.na(description))

prop_q6 <- prop_q6 |>
  mutate(desc_lower = str_to_lower(description),
         has_renovated = str_detect(desc_lower, "renovat"),
         has_shopping = str_detect(desc_lower, "shopping"),
         has_primary_school = str_detect(desc_lower, "primary school"),
         # Continuous time variable so the model can learn a price trend over
         # the years and project it forward to a future date. The (month-1)/12
         # term pins the prediction to a specific month (July 2026 = 2026.5), not
         # just "sometime in 2026".
         year_num = year(sold_time) + (month(sold_time) - 1) / 12)

# Cap area at the 99th percentile to limit a few giant blocks
area_cap <- quantile(prop$area, 0.99, na.rm = T)
# quantile lets the data tell you where the boundary is. It automatically scales to your dataset, whether you are measuring tiny apartments or massive commercial plots.
# Using the 99th percentile guarantees that you only flag exactly the top 1% of your absolute most extreme data points.

prop_q6 <- prop_q6 |>
  mutate(area = pmin(area, area_cap))
# pmin() (Element-wise Minimum): Looks at one row at a time, makes a choice for that specific row, and moves to the next. that is it compares the original area value of each individual row against your area_cap value, and it keeps whichever number is smaller.

cat("Training rows per suburb:\n")
```

```
## Training rows per suburb:
```

Hide

```
prop_q6 |> count(suburb)
```

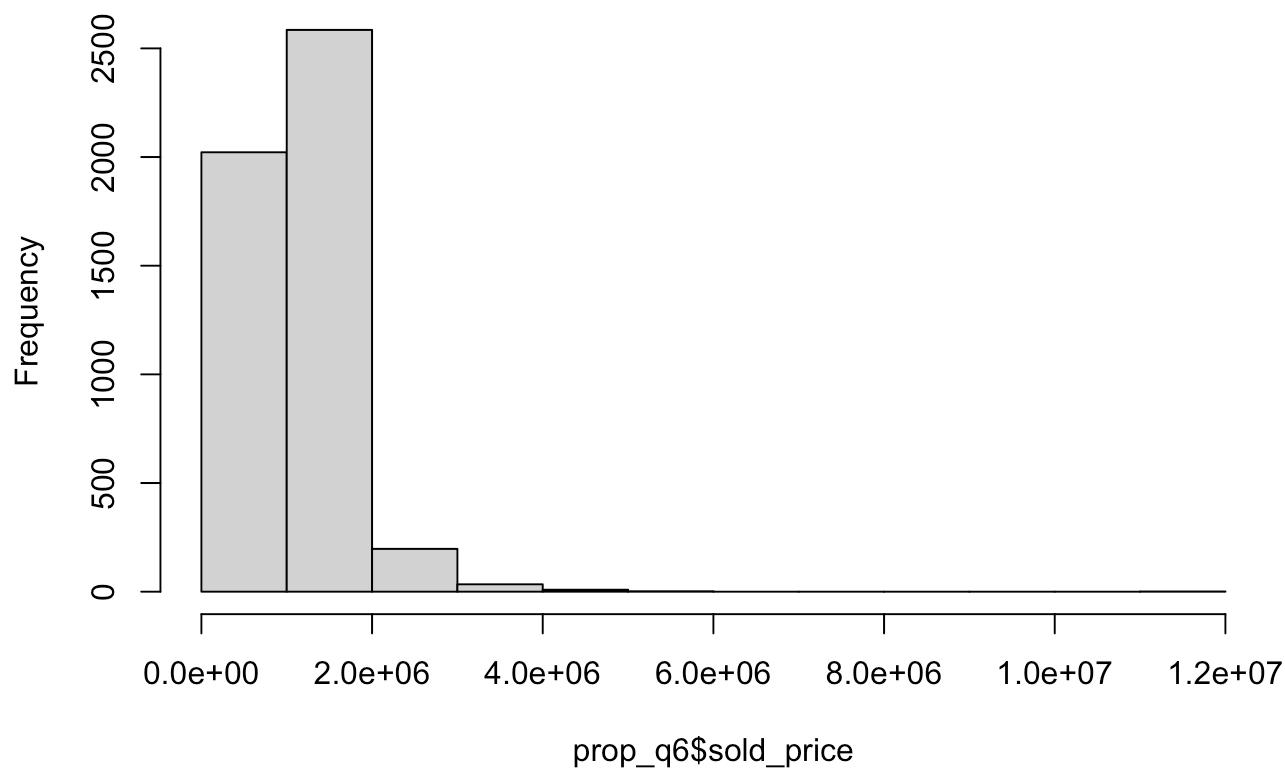
```
## # A tibble: 6 × 2
##   suburb      n
##   <chr>    <int>
## 1 Doncaster East    860
## 2 Glen Waverley   1077
## 3 Mulgrave        747
## 4 Rowville         870
## 5 Vermont South   460
## 6 Wheelers Hill    836
```

Hide

```
# Check skewness for choose a better Y(sold_price)
```

```
# Plot raw price
hist(prop_q6$sold_price, main = "Raw Price (Usually Skewed)")
```

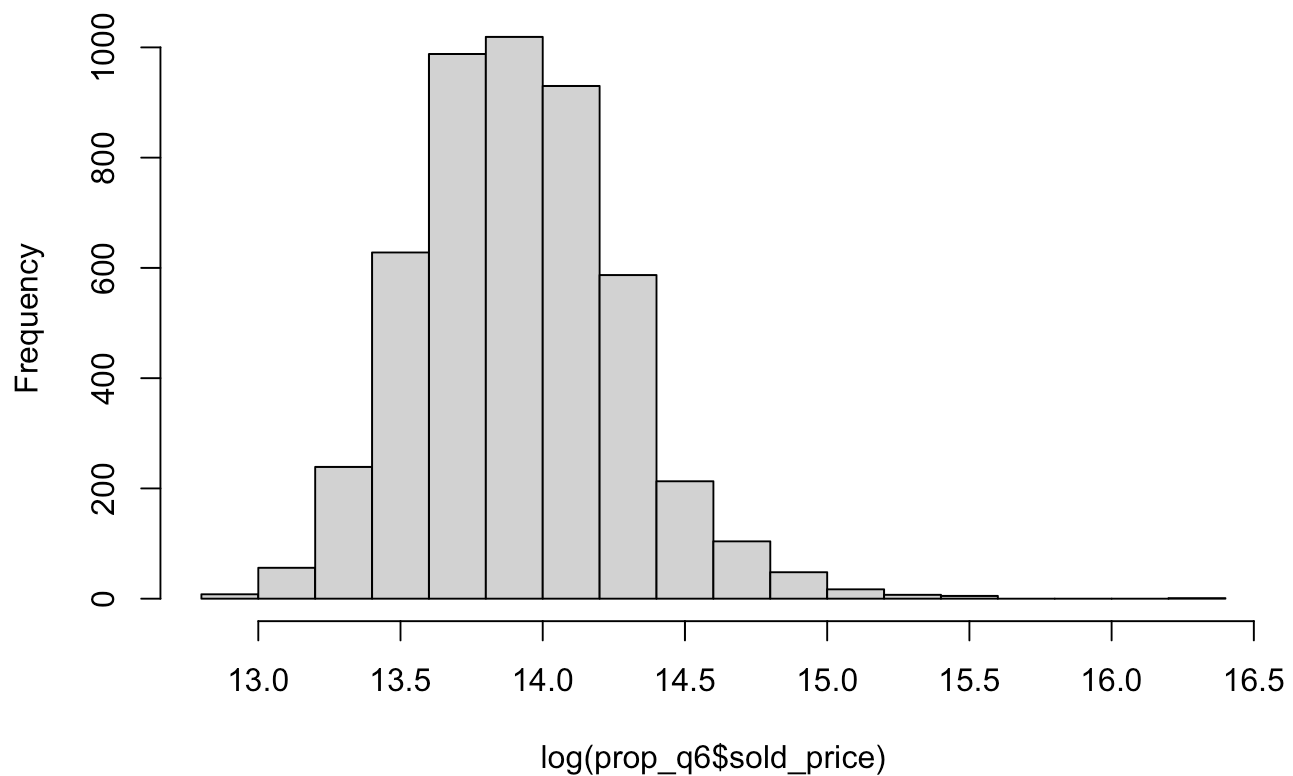
Raw Price (Usually Skewed)



Hide

```
# Plot log price
hist(log(prop_q6$sold_price), main = "Log Price (Usually Bell-Shaped)")
```

Log Price (Usually Bell-Shaped)



Raw `sold_price` is heavy right skew, so apply `log(sold_price)` in regression model.

[Hide](#)

```
model_q6 <- lm(log(sold_price) ~ suburb + beds + baths + parking + area +  
               year_num + has_renovated + has_shopping + has_primary_school,  
               data = prop_q6)  
  
summary(model_q6)
```

```
##
## Call:
## lm(formula = log(sold_price) ~ suburb + beds + baths + parking +
##     area + year_num + has_renovated + has_shopping + has_primary_school,
##     data = prop_q6)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.75131 -0.12279 -0.01064  0.10845  1.15475
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)    -1.038e+02  1.826e+00 -56.836 < 2e-16 ***
## suburbGlen Waverley    1.191e-01  8.883e-03  13.410 < 2e-16 ***
## suburbMulgrave    -2.784e-01  9.926e-03 -28.047 < 2e-16 ***
## suburbRowville    -3.884e-01  9.440e-03 -41.148 < 2e-16 ***
## suburbVermont South -1.121e-01  1.122e-02  -9.987 < 2e-16 ***
## suburbWhealers Hill -1.016e-01  9.510e-03 -10.680 < 2e-16 ***
## beds              4.998e-02  4.461e-03  11.203 < 2e-16 ***
## baths             1.136e-01  4.861e-03  23.368 < 2e-16 ***
## parking           8.296e-03  3.283e-03   2.527 0.01154 *
## area              3.660e-04  1.149e-05  31.864 < 2e-16 ***
## year_num          5.804e-02  9.061e-04  64.060 < 2e-16 ***
## has_renovatedTRUE  -1.827e-02  6.315e-03  -2.892 0.00384 **
## has_shoppingTRUE   -1.493e-02  6.297e-03  -2.371 0.01778 *
## has_primary_schoolTRUE -9.194e-03  6.116e-03  -1.503 0.13282
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.1918 on 4836 degrees of freedom
## Multiple R-squared:  0.7172, Adjusted R-squared:  0.7165
## F-statistic: 943.5 on 13 and 4836 DF,  p-value: < 2.2e-16
```

Hide

```
# Quick evaluate the model fitness
cat("Adjusted R-squared:", round(summary(model_q6)$adj.r.squared, 2), "\n")
```

```
## Adjusted R-squared: 0.72
```

Hide

```
cat("Training RMSE:", round(sqrt(mean(residuals(model_q6)^2)),2))
```

```
## Training RMSE: 0.19
```

Mind map

Imaging I am a staff of a real estate company, and Chris is my customer. He tells me his expectation, if I was him, I would like to see a table sheet. "suburb, predict price, the fluctuation", so that I can plan my financial budget.

Currently, I have built a good fit model, the questions are parking and area not be mentioned by Chris. When predict, it requires input specific value for each attribute. Hence take assumption firstly, filling with median value of them, which won't be distorted by outliers like mean.

[Hide](#)

```
# Plug in suburb's median value of area and parking into prediction model
# In order to remove impact of outliers, using Median instead of Mean.
typical <- prop_q6 |>
  filter( beds >= 4, baths >= 2) |>
  group_by(suburb) |>
  summarise(area = median(area),
            parking = median(parking),
            n_match = n(), # number of house meet these requirements
            .groups = "drop" # always remember to keep your data clean
  )

# Chris requirements
chris_input <- typical |>
  mutate(
    beds = 4, baths = 2,
    year_num = 2026 + 7/12, # July 2026, convert it to continue value
    has_renovated = T,
    has_shopping = T,
    has_primary_school = T)

# Prediction
log_pred <- predict(model_q6, chris_input, interval = "prediction", level = 0.8) # Assume
  Chris will consider +/- 20% fluctuation.

# Eventually output
answers <- chris_input |>
  mutate(predict_price = round(exp(log_pred[, "fit"])),
         predict_price_lower_80 = round(exp(log_pred[, "lwr"])),
         predict_price_higher_80 = round(exp(log_pred[, "upr"]))) |>
  arrange(desc(predict_price)) |> # assume Chris want to filter from expensive to cheap.
  select(suburb, predict_price, predict_price_lower_80, predict_price_higher_80, area, parking, n_match)
```

Answer

Table email to Chris if you are seller (predict_price_table_of_property_for_Chris_q6.png)

The 80% prediction intervals span roughly $\pm 20\%$, which is realistic of houses within a single suburb. The ordering matches the well-known pricing hierarchy of these SE Melbourne suburbs — Glen Waverley consistently commands a premium because of the high-demand school zone, while Mulgrave and Rowville sit at the more affordable end. >learn more details (<https://share.google/aimode/ID7TZSbj77HUr5Bov>)